

Daily Workflow

One interactive session owns a ticket through eight phases: brainstorm, spec, plan, implement, tests, docs, review, done. Every phase but the light one below writes a file under `.work/tickets/<id>/`, and two of them cannot be skipped by accident — `/crew:implement` refuses without an approved plan, and `/crew:done` refuses without a clean review receipt, a clean verify gate, and a passing completion audit.

For how the scope guard decides what a ticket may touch and what a report looks like when it refuses a write, see [daily-workflow-scope.md](#) (T3).

Full Path — Worked Example

1. Brainstorm. You type:

```
/crew:brainstorm add a --dry-run flag to the migrate command
```

The session mints T-0091, creates `.work/tickets/T-0091/`, and asks **one question at a time**:

Should `--dry-run` also print what it would do to files that already match (no-op writes), or only the ones that would actually change?

You answer. It proposes two approaches, recommendation first, then writes `.work/tickets/T-0091/direction.md` and stops. You say yes.

2. Spec. You type `/crew:spec T-0091`. The session reads the direction, uses `crew:explorer` to pin down `path:line` evidence, and writes `spec.md` with Intent, Exclusions, Evidence, Unknowns, Touch and Acceptance. `.work/INDEX.md` gets a row.

3. Plan. You type `/crew:plan T-0091`. The session reads `spec.md`, writes one step per unit of work in `plan.md` (Files/Test/Risk/Standards each), checks every Files: entry against the spec's Touch globs, and enters **plan mode** to show you the whole thing. You review it. On your yes, it asks you to type `/crew:approve T-0091`. crew's prompt hook sees that you typed it and writes the approval receipt, bound to this plan's exact contents — edit `plan.md` after this and the receipt no longer matches.

4. Implement. You type `/crew:implement T-0091`. First thing it does: checks the receipt. If you skipped step 3, or edited the plan after approving it, it refuses here and tells you which. Assuming it passes, it records the scope base (`scope_base.py --record`), works the plan step by step — test first, watch it fail, implement, watch it pass — and the **plan-approval + scope guard hook** blocks any write outside the spec's Touch globs before it happens, not after.

5–6. Tests and docs. Coverage lands as part of implementing the plan's steps. `/crew:docs` runs next and usually says "none" — most tickets touch no document that needs updating.

6b. The standards self-check. Before the review, the session answers every development standard in the effective set (crew's generic GEN standards, any per-language set a changed file matches, and this repository's `.crew/standards.md` overlay) in `.work/tickets/T-0091/selfcheck.md`: addressed with evidence, or n/a with a reason. `crew_standards.py stamp` refuses an incomplete answer sheet and binds a complete one to the exact review bundle, so any later edit needs a new stamp.

7. Review. `/crew:implement` calls `/crew:review T-0091` last, after tests, docs and the self-check; without a current stamp the review refuses before it spends a round. Codex reviews the bundle (or Copilot, or the Claude fallback, whichever survives the author-family strike), reports BLOCK/FIX/NIT lines, and you fix the BLOCKs. Two rounds total, ticket-wide — a third is refused and the ticket becomes NEEDS_REPLAN. Each round's findings go into a `standards-proposals-r<N>.md` file, one row per finding; you approve or reject each proposed standard, and nothing is added to a standards file on its own.

With parallel lanes in one clone, arm its **merge train** once (`crew_train.py arm`, L-0520). Lanes still implement at the same time, overlapping Touch or not; only gate and land queue. Before the review round, `crew_train.py acquire --ticket T-0091` takes the train: a ticket whose Touch overlaps one already holding it waits (exit 1, colliding paths named) and gates next, in the order the lanes reached their gate, while a ticket with a disjoint Touch gates at once. Catch up with `crew_train.py catch-up --ticket T-0091` - a git merge of the base, never a rebase, with git rerere on so a conflict resolved once replays next time. A replay is left unstaged and listed: inspect it, `git add` it, and show it to the reviewer. crew never turns on `rerere.autoupdate`, and the version files (`plugin.json`, `marketplace.json`, `PLUGINS.md`, `CHANGELOG.md`) are never replayed - they come back conflicted, named as forgotten, for you to resolve by hand.

8. Done. You type `/crew:done T-0091`. Three checks, all required: the review receipt rebuilds clean, the verify gate is clean, and the completion audit (the whole tree diffed against the scope base) finds nothing outside scope. Any one failing refuses the close and names what to fix. On

success it appends a metrics row, marks the ticket done, and clears a stale handoff.

With the train armed, landing is part of done: `crew_train.py check-land --ticket T-0091 --pr <n>` refuses unless T-0091 holds the train, `git merge-tree` against the base is clean, the base has not moved in Touch paths, and HEAD carries the review receipt and a green gate; then it prints `gh pr merge <n> --merge --match-head-commit <sha>` for you to run (crew never merges). After the merge, `crew_train.py release --ticket T-0091 --merged <merge sha>` frees the train and tells every overlapping lane to merge the base now.

What Each Hook Does, in Order

When	Hook	Does
Every write	plan-approval + scope guard (PreToolUse)	refuses an edit with no approval receipt, or outside the spec's Touch
End of turn	verify gate (Stop)	refuses to end the turn on a red check
End of turn	completion scope audit (Stop)	diffs the whole tree against the scope base, catches shell-made writes too
Session start	context	brief on branch, open ticket, gate state, codemap freshness

None of these ask you anything mid-turn — a blocked write or a red gate shows up as a refusal with a reason, not a prompt.

Light Path — `/crew:fix`

For a small, well-understood change (one subsystem, no new behaviour, known cause, nothing touching auth/SQL/IaC/secrets/migrations):

```
/crew:fix the changed-file list in implement.md prints nothing on an empty diff
```

Every phase still runs, compressed:

- **Direction:** one line, not a question round — Fix: <what you typed>.

- **Spec:** six lines — Intent, Touch, Acceptance — Exclusions, Evidence and Unknowns omitted, not forgotten.
- **Plan:** exactly one step, still shown, still needing the approval receipt.
- **Implement, tests, docs:** same as the full path, sized to one step.
- **Review:** one Codex round instead of two.
- **Done:** the same three checks — the light path does not skip its own gate.

If the change turns out to be bigger than it looked — a second subsystem shows up, the cause wasn't what you thought — the session says so and hands off to `/crew:spec <id>` to fill in what the light path left out, rather than forcing the rest of the ticket through a path that no longer fits it.

Web Testing — `/crew:webtest`

For a ticket whose acceptance criteria describe UI behaviour (the stack-web skill triggers on `playwright.config.*`, `@playwright/test`, or an Angular e2e suite), `/crew:webtest` sits inside the normal eight phases rather than replacing them:

1. **Planner.** `playwright-test-planner` reads the ticket's acceptance criteria and writes `specs/<ticket>.md` — what to test, not code yet.
2. **Plan.** crew's own `/crew:plan` reads that spec the same way it reads any other Files/Test/Risk plan, and needs the same approval receipt before implement starts.
3. **Generator.** `/crew:implement` calls `playwright-test-generator` against the approved spec to write the actual `*.spec.ts` files.
4. **Healer.** A failing generated test goes to `playwright-test-healer` for locator/wait repairs only. **A Healer "skip" is a finding, not a pass** — it means the healer decided the feature itself is broken, and review is where that surfaces rather than sailing through as a green suite.
5. **Review.** The reviewer gets the trace zip (`trace: 'on-first-retry'`) and the axe accessibility attachment alongside the diff, the same way it gets the completion audit today.

Visual regression (`toHaveScreenshot`) only runs inside `mcr.microsoft.com/playwright:v1.63.0-noble` — a baseline captured on a bare host does not match CI's font hinting and subpixel rendering, and fails every later run for reasons unrelated to the change under test. Outside that

image the visual rule reports **UNVERIFIED**, not a pass; see [Troubleshooting](#).

If Something Refuses

Refusal	Means	Do
<code>/crew:implement</code> says no approved plan	step 3 was skipped, or <code>plan.md</code> changed after approval	<code>/crew:plan <id></code> , then type <code>/crew:approve <id></code>
a write is blocked outside Touch	the file isn't in the spec's declared scope	amend <code>spec.md</code> 's Touch and re-approve the plan, or don't make the edit
<code>/crew:review</code> exits 2 with <code>review-run:</code> <code>self-check: ...</code>	the standards self-check is missing, incomplete, or stamped for an earlier state of the change	answer <code>.work/tickets/<id>/selfcheck.md</code> , run <code>crew_standards.py stamp --root . --</code> <code>ticket <id></code> , rebuild the bundle and run the round again; no round was spent
<code>/crew:done</code> reports <code>NEEDS_REPLAN</code>	the review budget (two rounds) is spent	<code>/crew:plan <id></code> for a successor plan; no third round
<code>/crew:done</code> fails the completion audit	a path outside scope changed, including one a shell command wrote	file it to <code>TOD0.md</code> , not to this ticket, then rerun

See [Troubleshooting](#) for the rest.

Scope and Approval

Every crew 1.0 ticket has a contract: the spec says which files it may touch, the plan says which of those each step changes, and you approve both. Two hooks then hold the session to that contract. This section explains what they do, what they print, and how to change scope while you work.

The Contract

A ticket lives in `.work/tickets/<id>/`:

- `spec.md` has six sections: `## Intent`, `## Exclusions`, `## Evidence`, `## Unknowns`, `## Touch` and `## Acceptance checks`. `## Touch` lists one repo-relative glob or path per bullet:

```
## Touch

- `src/export/**`

- `tests/test_export.py`

- `docs/export.md`
```

- `plan.md` is a list of steps. Each step has a `Files:` line, a `Test:` line and a `Risk:` line. Every `Files:` entry must fall inside `Touch`. A plan cannot add a path the spec does not list.

Check the contract at any time:

```
python3 <crew>/hooks/scripts/crew_ticket.py validate --ticket T-0042
```

A plan path outside `Touch` is reported as `INVALID`: `plan Files entry '...' is outside spec ## Touch`. To fix it, add the path to `Touch`. `crew` never widens `Touch` for you.

Approval

When the plan is ready, the session asks you to approve it. Type this yourself:

```
/crew:approve T-0042
```

`crew`'s `UserPromptSubmit` hook sees the prompt you typed, validates the contract, then records the sha256 of `spec.md` and `plan.md` in `<git-common-dir>/crew/tickets/T-0042/approval.json`, marked `approved_via: "user-prompt"`. If the contract does not validate, the hook blocks the prompt and says why. That file is outside the worktree, and the scope guard refuses any `Write` or `Edit` to it and any shell command that runs `crew_ticket.py approve`, so the session cannot approve its own plan. The one exception is opt-in: with `scope.allowCliApproval: true` and `autopilot.approval` set to `self` (or `risk`, for a `risk: low spec`), `/crew:autopilot` runs `crew_autopilot.py approve`. It writes what every approval route writes: `approval.json` (marked `approved_via: "autopilot"`), `scope-tickets.json` on a ticket's first approval, and, for a distinct successor plan under a spent review budget, the review ledger moved `NEEDS_REPLAN` -> `IN_REVIEW`. Both default off, so out of the box nothing self-approves. The approval is a step you take, not a lock.

`crew_ticket.py status --ticket T-0042` prints one of three states:

- approved: both files match the receipt.
- stale: `spec.md` or `plan.md` changed since approval.
- none: the ticket has never been approved.

While You Implement: The Scope Guard

Before every Write, Edit, MultiEdit or NotebookEdit, the guard checks the target file:

- The ticket's own `.work/tickets/<id>/` files are always allowed, so you can amend the spec and plan.
- Any other file needs a current approval and a path inside Touch.
- The code map, the diagrams dir, the graph dir and `.claude/rules/` are writable for an approved ticket, because `/crew:implement` step 6 refreshes them.
- Nothing else is exempt: `.crew/` beyond the code map, `TODO.md`, the rest of `.claude/` and crew's own policy files. If the ticket changes one of them, list it in Touch.
- Approval receipts, the review ledger and the scope base are always refused.

The guard checks the real file a symlink points to and the path as written, so a link cannot move a write into or out of scope. `..` is resolved the way the operating system resolves it. Refusals look like this:

```
SCOPE GUARD: refused Edit on other/keep.py.
```

```
Reason: other/keep.py is outside T-0042's spec ## Touch.
```

```
To widen scope: amend .work/tickets/T-0042/spec.md ## Touch (and plan.md), then  
ask the user to type
```

```
`/crew:approve T-0042`. (scope.mode is block)
```

At the End of a Turn: The Completion Audit

The guard only sees the editing tools. A `sed -i`, a shell redirect or a formatter can still change files. So when the session stops, the completion audit compares the whole tree with the commit the ticket started from. It covers committed, staged, unstaged and untracked changes, and both ends of a rename. If any changed path is outside Touch, the audit blocks the stop and lists the

paths in six lines or fewer. It never blocks the continuation it caused.

A changed refresh artifact passes the audit only when a path the ticket changed reaches it and the edit is a re-anchor (the anchor: or provenance sha moved forward to a commit on this branch) or a regeneration (.claude/rules/ as crew_instructions.py rules writes them, the graph after a code change). A deleted rendered diagram or graph file, a symlink at or along an artifact's path, or a file whose git mode changed never passes. A map claim edited without a re-anchor is listed with [anchor did not move], and belongs in Touch if that is what the ticket means to do. When git cannot answer, a rule file cannot be read, a short anchor is ambiguous (two commits share it), the artifact dirs cannot be resolved, or a directory the hook cannot search hides whether the config, a rule or a map exists, or two configured artifact dirs are equally specific for a path, the listing says [could not tell: ...], and that never passes. (These verdicts are crew_refresh_check.py's; the audit applies them once L-0540 lands, and until then admits the artifact dirs for an approved ticket.) /crew:done runs the same check:

```
python3 <crew>/hooks/scripts/completion_audit.py --check --ticket T-0042
```

The audit sees what git sees. Gitignored files, including everything under .crew/, and .work/ are outside it.

Amending Scope

1. Edit spec.md ## Touch to add the path. Update plan.md if a step now changes it.
2. Run crew_ticket.py validate --ticket <id>.
3. Type /crew:approve <id> yourself.

Between steps 1 and 3 the approval is stale. The guard refuses edits outside the ticket directory until you approve again.

After Two Review Rounds: A Successor Plan

If a ticket uses both review rounds without an accepted receipt (refunded tool-failure rounds do not count), it moves to NEEDS_REPLAN. Write a different plan and approve it. approve reports review may continue, and the ledger gives the new plan two fresh rounds. Approving the same plan again is refused and exits with status 3.

Modes

Set `scope.mode` in `.crew/config.json`:

Value	What happens
<code>off</code>	Default. Neither hook does anything.
<code>report</code>	Everything is allowed. Would-be refusals go to <code>.crew/guard.log</code> and appear as a system message.
<code>block</code>	Refusals block.
<code>auto</code>	report for the first ten tickets approved in the repository, then <code>block</code> .

If the config file does not parse, or `scope.mode` has an unknown value, crew treats it as `block` and says so.

Which Ticket Is Active

`crew_ticket.py activate --ticket <id>` sets the active ticket for the current worktree. `deactivate` clears it, and `active` prints it. Without a setting, crew uses the open ticket in `.work/INDEX.md` if `.work/tickets/<id>/` exists. With no active ticket, the edit guard only protects approval and ledger state, and the completion audit does nothing.

Who moves it depends on whether the move can widen scope. `/crew:autopilot` sets it itself when the worktree has no active ticket, when it already names the ticket, or when it names a ticket closed by an accepted review (a `clean` or `owner-accepted` receipt in the review ledger, plus done in `.work/INDEX.md` or the spec header). Moving off a ticket that is still in flight is yours: type `/crew:autopilot <id>`, and crew's prompt hook re-points this worktree and says which ticket it left. The edit guard refuses the session's own `activate` or `deactivate` for that move under every mode but `off`, and autopilot's stops recommend finishing the in-flight ticket rather than telling you to run a script. A review ledger that says `accepted` but whose receipt does not stand on its latest completed round, and a `--root` the guard cannot judge where the shell will (`~`, a variable, a backslash path, a directory change, or no repository where it looked), both count as could not tell, so the move stays yours. If the pointer moves between the prompt hook reading it and writing it, the hook writes nothing and says so.